

```
module instr_mem_tb;
```

```
    // Testbench signals
```

```
    reg clk;
```

```
    reg rst_n;
```

```
    reg [31:0] i_addr;
```

```
    reg i_stb;
```

```
    wire o_ack;
```

```
    wire [31:0] o_data;
```

```
    // Instantiate the instruction memory module
```

```
    instr_mem uut (
```

```
        .clk(clk),
```

```
        .rst_n(rst_n),
```

```
        .i_addr(i_addr),
```

```
        .i_stb(i_stb),
```

```
        .o_ack(o_ack),
```

```
        .o_data(o_data)
```

```
    );
```

```
    // Clock generation
```

```
    always #5 clk = ~clk;
```

```
    initial begin
```

```
        // Initialize signals
```

```
        clk = 0;
```

```
        rst_n = 0;
```

```
        i_addr = 32'd0;
```

```
        i_stb = 0;
```

```
        // Reset sequence
```

```
        #10 rst_n = 1;
```

```
        // Test Case 1: Read instruction at address 0
```

```
        #10 i_stb = 1; i_addr = 32'h00000000;
```

```
        #10 i_stb = 0;
```

```
        // Test Case 2: Read instruction at address 4 (next instruction)
```

```
        #10 i_stb = 1; i_addr = 32'h00000004;
```

```
        #10 i_stb = 0;
```

```
        // Test Case 3: Unaligned address (should trigger a warning)
```

```
        #10 i_stb = 1; i_addr = 32'h00000003;
```

```
        #10 i_stb = 0;
```

```
        // Test Case 4: Read another instruction
```

```
#10 i_stb = 1; i_addr = 32'h00000008;
```

```
#10 i_stb = 0;
```

```
// Test Case 5: Atomic instruction read (example AMO operation)
```

```
#10 i_stb = 1; i_addr = 32'h0000000C;
```

```
#10 i_stb = 0;
```

```
// Finish simulation
```

```
#20 $finish;
```

```
end
```

```
endmodule
```